FOORAN

GPIO and Interrupt PROGRAMMING USING C

STM32F103RB Microcontroller Hardware Abstraction Layer (HAL)

Prepared By: Ahsan Basharat | Status: Intern | Submitted To: Sir Yasir

Objectives

- To understand the fundamental architecture and operational modes of General Purpose Input/Output (GPIO) peripherals in STM32 microcontrollers.
- To configure GPIO pins as Digital Outputs to drive external devices such as Light Emitting Diodes (LEDs).
- To configure GPIO pins as Digital Inputs with internal Pull-Up resistors to interface with momentary push buttons.
- To utilize STM32 Hardware Abstraction Layer (HAL) API functions for writing, reading, and toggling pin states.
- To gain hands-on experience in building, flashing, and debugging embedded C projects within the STM32CubeIDE environment.

Introduction

General Purpose Input/Output (GPIO) pins serve as the primary bridge connecting microcontrollers to external electronic components and sensors. In embedded systems, GPIO pins can be configured via software registers to operate in Digital Input mode (sensing physical button states or logic levels) or Digital Output mode (controlling LEDs, relays, or digital signals). Mastering GPIO programming using the STM32 Hardware Abstraction Layer (HAL) provides a foundation for developing robust real-time embedded applications.

GPIO as an External Interrupt

GPIO can be configured as an External Interrupt (EXTI) to detect events such as a button press without continuously checking the pin state. When the configured GPIO pin detects a rising edge, falling edge, or both, an interrupt request is generated. The processor temporarily pauses the current task and executes the interrupt service routine (ISR). After the ISR finishes, the processor returns to the main program. This method improves efficiency and enables fast response to external events.

Required Components

#	Component / Tool	Quantity	Description / Specifications
1	STM32F103RB	1	ARM Cortex-M3 (32-bit)
2	USB Cable / ST-Link V2	1	Hardware programmer and debugger
3	Breadboard	1	Standard solderless prototyping breadboard
4	Light Emitting Diode (LEDS)	4	5mm Red/Green LED indicator
5	Resistor (220 Ω)	1	Current-limiting resistor for LED circuit
6	Jumper Wires	Set	Male-to-Male and Male-to-Female wires

7	STM32CubeIDE/MX	Software	Integrated Development Environment for C/C++

Circuit Description

• **LED Connection:** The anode of the LED is connected to GPIO pin PA5 on the STM32 board, while the cathode is connected to Ground (GND) through a 220 Ω current-limiting resistor. Writing logic HIGH (GPIO_PIN_SET) to PA5 drives current through the LED, turning it ON.
• **User Button Connection:** One terminal of the tactile push button is connected to GPIO pin PC13, and the opposite terminal is connected to GND. Pin PC13 is configured in software with an internal Pull-Up resistor. When the button is unpressed, PC13 reads logic HIGH (GPIO_PIN_SET). When pressed, the switch pulls PC13 to GND, resulting in a logic LOW (GPIO_PIN_RESET) reading.

Procedure

1. Launch STM32CubeIDE and create a new project selecting the target MCU: STM32F103C8T6.

2. In the Device Configuration Tool (.ioc Pinout view), configure pin PA5 as GPIO_Output.

3. Configure pin PC13 as GPIO_Input, and in the Configuration panel, set its GPIO Pull-Up/Pull-Down setting to 'Pull-Up'.

4. Assign User Labels: set PA5 as 'BOARD_LED' and PC13 as 'USER_BTN'.

5. Save the IOC file to auto-generate the peripheral initialization code.

6. Implement code using HAL_GPIO_ReadPin() to check the push button state on PC13.

7. Use HAL_GPIO_WritePin() or HAL_GPIO_TogglePin() to turn the LED on PA5 ON or OFF based on button inputs.

8. Connect the ST-Link V2 debugger to your PC and target board, then flash the project ('Run'). Test operational hardware behavior.

HAL Functions Used

Function Name	Purpose
`HAL_GPIO_Init()`	Initializes the GPIO peripheral according to the specified parameters in the GPIO_InitTypeDef structure.
`HAL_GPIO_WritePin(LEDS *const led)`	Sets or clears the logical state of a targeted GPIO output pin (GPIO_PIN_SET or GPIO_PIN_RESET).
`HAL_GPIO_ReadPin(LEDS *const led)`	Reads and returns the digital logical state of a specified GPIO input pin.
`HAL_GPIO_TogglePin(LEDS *const led)`	Toggles / inverts the current logical state of the specified GPIO pin.
`LED_On(LEDS *const led)`	Turns the specified LED ON by setting its GPIO pin.
`LED_Off(LEDS *const led)`	Turns the specified LED OFF by resetting its GPIO pin.
`GPIO_EXTI_Init(LEDS *const led);`	Configures the specified GPIO pin as an **External Interrupt (EXTI)** input.
`GPIO_EXTI_Callback(uint16_t GPIO_Pin)`	Executes user-defined code when an external interrupt occurs on the specified GPIO pin.

Code:

```c
#include "stm32f1xx_hal.h"

/*==========================================================
 * GPIO Object
 *==========================================================*/
typedef struct
{
    GPIO_TypeDef *Port;
    uint16_t Pin;

} LEDS;

/*==========================================================
 * Function Prototypes
 *==========================================================*/

/* Initialize GPIO Pin as Output */
void LED_Init(LEDS * const led);

/* LED Control */
void LED_On(LEDS * const led);
void LED_Off(LEDS * const led);
void LED_TogglePin(LEDS * const led);

/* Read Pin State */
GPIO_PinState Read_Pin(LEDS * const led);

#endif /* INC_GPIO_HAL_H_ */
```

```c
/*----------------------------------------------------------
 * Initialize GPIO Output
 *----------------------------------------------------------*/
void LED_Init(LEDS * const led)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    if (led == NULL)
    {
        return;
    }

    /* Enable GPIO Clock */
    if (led->Port == GPIOA)
    {
        __HAL_RCC_GPIOA_CLK_ENABLE();
    }
    else if (led->Port == GPIOB)
    {
        __HAL_RCC_GPIOB_CLK_ENABLE();
    }
    else if (led->Port == GPIOC)
    {
        __HAL_RCC_GPIOC_CLK_ENABLE();
        __HAL_RCC_AFIO_CLK_ENABLE();
    }
    else
    {
        return;
    }

    /* Configure PC13 as External Interrupt */
    if ((led->Port == GPIOC) && (led->Pin == GPIO_PIN_13))
    {
        GPIO_InitStruct.Pin  = led->Pin;
        GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
        GPIO_InitStruct.Pull = GPIO_PULLUP;

        HAL_GPIO_Init(led->Port, &GPIO_InitStruct);

        HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0, 0);
        HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
    }
    else
    {
        /* Configure LED Output */
        GPIO_InitStruct.Pin   = led->Pin;
        GPIO_InitStruct.Mode  = GPIO_MODE_OUTPUT_PP;
        GPIO_InitStruct.Pull  = GPIO_NOPULL;
        GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;

        HAL_GPIO_Init(led->Port, &GPIO_InitStruct);

        HAL_GPIO_WritePin(led->Port,
                          led->Pin,
                          GPIO_PIN_RESET);
    }
}
```

```c
/*----------------------------------------------------------
 * LED ON
 *----------------------------------------------------------*/
void LED_On(LEDS * const led)
{
    if (led != NULL)
    {
        HAL_GPIO_WritePin(led->Port,
                          led->Pin,
                          GPIO_PIN_SET);
    }
}

/*----------------------------------------------------------
 * LED OFF
 *----------------------------------------------------------*/
void LED_Off(LEDS * const led)
{
    if (led != NULL)
    {
        HAL_GPIO_WritePin(led->Port,
                          led->Pin,
                          GPIO_PIN_RESET);
    }
}
```

```c
/*----------------------------------------------------------
 * Toggle LED
 *----------------------------------------------------------*/
void LED_TogglePin(LEDS * const led)
{
    if (led != NULL)
    {
        HAL_GPIO_TogglePin(led->Port,
                           led->Pin);
    }
}

/*----------------------------------------------------------
 * Read Pin State
 *----------------------------------------------------------*/
GPIO_PinState Read_Pin(LEDS * const led)
{
    GPIO_PinState State = GPIO_PIN_RESET;

    if (led != NULL)
    {
        State = HAL_GPIO_ReadPin(led->Port,
                                 led->Pin);
    }

    return State;
}
```

```c
#ifndef INC_INTERR_FUN_H_
#define INC_INTERR_FUN_H_

#include "GPIO_HAL.h"
#include "stm32f1xx_hal.h"

void GPIO_EXTI_Init(LEDS * const led);
void GPIO_EXTI_Callback(uint16_t GPIO_Pin);

#endif /* INC_INTERR_FUN_H_ */
```

```c
void GPIO_EXTI_Init(LEDS * const led)
{
    GPIO_InitTypeDef GPIO_InitStruct = {0};

    if (led == NULL)
    {
        return;
    }

    /* Enable GPIO Clock */
    if (led->Port == GPIOA)
    {
        __HAL_RCC_GPIOA_CLK_ENABLE();
    }
    else if (led->Port == GPIOB)
    {
        __HAL_RCC_GPIOB_CLK_ENABLE();
    }
    else if (led->Port == GPIOC)
    {
        __HAL_RCC_GPIOC_CLK_ENABLE();
    }
    else
    {
        return;
    }

    GPIO_InitStruct.Pin  = led->Pin;
    GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
    GPIO_InitStruct.Pull = GPIO_PULLUP;

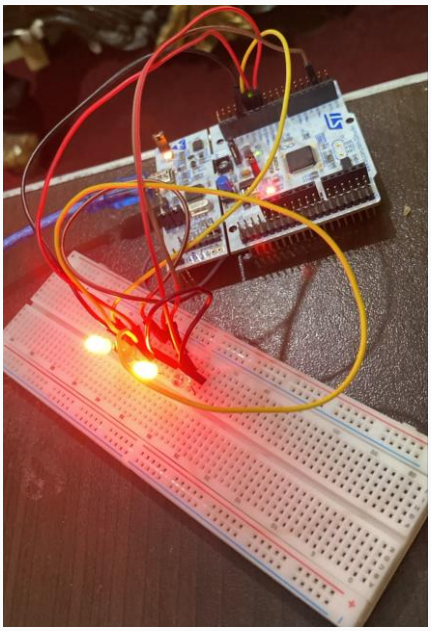
    HAL_GPIO_Init(led->Port, &GPIO_InitStruct);

    HAL_NVIC_SetPriority(EXTI15_10_IRQn, 0U, 0U);
    HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
}
```

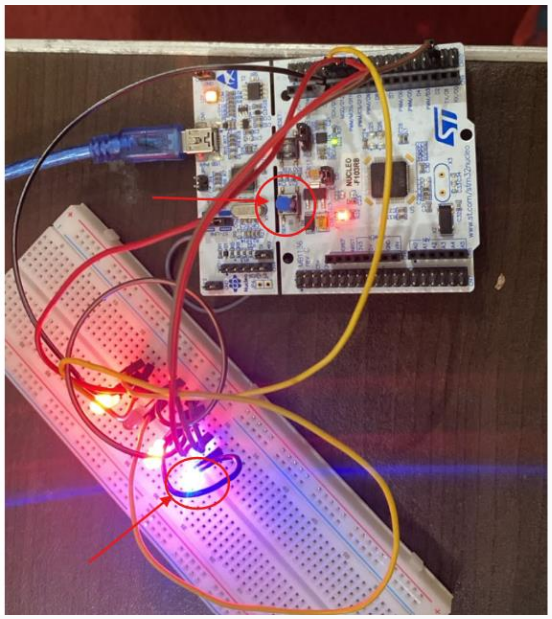
```c
}
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
    if(GPIO_Pin==GPIO_PIN_13){
        LED_TogglePin(&LED3);
        LED_TogglePin(&LED2);
    }
}
```

Program Output

Simple GPIO Leds on off with HAL_Delay(1000)



In this Output PC13 set a External Interrupt and set Mode a Rising edge Trigger



Result

Initial Condition:

- LED connected to PA5 is ON (HIGH).
- LED connected to PA6 is ON (HIGH).
- LED connected to PA7 is OFF (LOW).

After External Interrupt (PC13 User Button Pressed):

- The interrupt callback function is executed immediately.
- PA5 remains ON (HIGH).
- PA6 remains ON (HIGH).
- PA7 changes to ON (HIGH), indicating that the interrupt has been successfully detected and handled.

Conclusion

This experiment demonstrated the configuration and use of GPIO as both an output and an external interrupt using the STM32 HAL library. The GPIO output was used to control an LED, while the external interrupt detected a push-button event and executed the corresponding callback function. The experiment also introduced the use of common HAL GPIO functions for pin initialization, reading input states, writing output states, and handling interrupts.

Furthermore, the code was developed following MISRA C coding guidelines to ensure better readability, maintainability, reliability, and safety. Following these guidelines helps reduce programming errors and encourages the development of high-quality embedded software for STM32-based applications.

Github Repositary:

https://github.com/Ahsan-web-hue/GPIO-Interrupt-STM32-Nucleo-F103RB.git

https://github.com/Ahsan-web-hue/GPIO-External-Interrupt-EXTI-using-STM32-Nucleo-F103RB.git